Разработка ваших первых программ

При написании программы у вас, как правило, есть какая-то проблема, которую нужно решать. Новичок очень часто спешается на этапе преобразования идеи решения проблемы в реальный код. Но самое главное, что вам нужно запомнить — разработка программы выполняется перед этапом написания кода.

Во многих отношениях, программирование — это как архитектура. Что происходит, если вы попытаетесь построить для без соблюдения архитектурного плана? Для может вы и сумеете построить, но какой он будет: кривые стены, протекающая крыша и т.д. Аналогично, если вы начнете программировать, что необходимо сделать, прежде чем, как составлен план, то очень скоро обнаружите, что ваш код имеет очень много проблем, на решение которых вы потратите гораздо больше временнусловий нервов, нежели на изначальное составление хорошего плана.

Оглавление:

- Шаг №1: Определите проблему
- Шаг №2: Определите свой инструментарий, цели и план бюкала
- Шаг №3: Разбейте проблему на части
- Шаг №4: Определите последовательность событий
- Шаг №5: Определите данные ввода и данные вывода на каждом этапе
- Шаг №6: Позаботьтесь о деталях
- Шаг №7: Соедините данные ввода с данными вывода в программе
- Советы

Шаг №1: Определите проблему

Первое, что вам нужно сделать — это определить проблему, которую решает ваша программа. В идеале, вы должны сформулировать это одним или двум предложениям. Например:

- Я хочу написать программу-справочник для удобного хранения и редактирования всех телефонных номеров и звонков.
- Я хочу написать программу для генерации случайных чисел, с помощью которой можно будет определить победителей разных конкурсов.
- Я хочу написать программу, которая будет отслеживать и анализировать акции на фондовом рынке для генерации выгодных прогнозов.

Хотя этот шаг кажется очевидным, но он также очень важен. Самое худшее, что вы можете сделать — это написать программу, которая делает не то, что вам нужно!

Шаг №2: Определите свой инструментарий, цели и план бюкала

Для опытных программистов на этом этапе будет еще немало дополнительных пунктов:

- Какая ваша целевая аудитория и какие у нее потребности?
- На какой операционной/ОС ваша программа будет работать?
- Какой инструментарий вы будете использовать?
- Будете ли вы разрабатывать программу в одиночку или в составе команды?
- Анализ требований.
- Определение стратегий тестирования/отладкой связи/релиза.
- Создание плана бюкала в случае неожиданных проблем.

Новичок, как правило, большим количеством вопросов не задается: «Язык программу для собственного использования, в одиночку, на своей операционной системе, с помощью своей IDE, пользуясь этой программой буду только я». Всё просто. Если же вы будете работать над чем-нибудь посерьезнее, то стоит ещё задаваться и плана бюкала вашей программы/проекта. Это не просто тетрадь с кодом в другом углу ноутбука (хотя это уже фуже, чем ничего). Если вашы ОС «слежается», то вы потеряете все данные. Хорошая стратегия резервного копирования включает в себя создание копии вашего кода вне вашей операционной системы, например:

- Отправить свою себе E-mail с кодом (прикрепить как файл).
- Скопировать в Dropbox или в любое другое облако.
- Перенести на внешнюю запоминающее устройство (например, на портативный жёсткий диск).
- Скопировать на другой компьютер в локальной сети.
- Воспользоваться системами контроля версий (например, GitHub, GitLab или Bitbucket).

Шаг №3: Разбейте проблему на части

В реальной жизни чаще приходится вытаскивать очень сложные задачи. Понять, как их решить, также бывает очень трудно. В таких случаях можно использовать **метод деления на части** или **метод «от большого к малому»**. То есть, вместо того, чтобы решать одну большую сложную задачу, мы разбиваем её на несколько поддач, каждую из которых проще решить. Если же поддачи все еще слишком сложны, то также нужно еще раз разбить. И так до тех пор, пока мы не доберемся до точки, где каждая отдельно взятая задача — легко решаема.

Рассмотрим это на примере. Допустим, что нам нужно написать доклад о картошке. Наша иерархия задач в настоящем времени выглядит следующим образом:

- Написать доклад о картошке

Это довольно большая задача, давайте разделим её на поддачи:

- Написать доклад о картошке
  - Поиск информации о картошке
  - Создание плана
  - Заполнение каждого пункта плана подобной информацией
  - Заключение

Это уже проще, так как теперь мы имеем список поддач, на которых можно сосредоточиться в индивидуальном порядке. Тем не менее, в данном случае, «Поиск информации о картошке» звучит немного расплывчато, нужно дополнительно разбить и этот пункт:

- Написать доклад о картошке
  - Поиск информации о картошке
    - Сходить в библиотеку за книжками о картошке
    - Поискать информацию в Интернете
    - Делать заметки в соответствующих разделах из странового материала
  - Создание плана
    - Информация о выращивании
    - Информация об обработке
    - Информация об удобрениях
  - Заполнение каждого пункта плана подобной информацией
  - Заключение

Выполняя каждый подпункт этого списка, мы решим одну большую задачу.

Есть еще один метод создания иерархии — **метод «от малого к большому»**. Рассмотрим на примере.

Большинство из нас выполняет задачи на работу (школу/университет) по будиям. Предположим, что нам нужно решить проблему «Я опаздываю на работу». Если вы, как спросили, что вы делаете перед тем, как добраться на работу, вы бы ответили примерно следующее:

- Выбрать одежду
- Одеться
- Позавтракать
- Ехать на работу
- Почистить зубы
- Встать с постели
- Приготовить завтрак
- Принять душ

Используя метод «от малого к большому», мы можем сгруппировать задачи и создать иерархию:

- От постели к работе
  - Спальня
    - Встать с постели
    - Выбрать одежду
    - Одеться
  - Ванная
    - Принять душ
    - Почистить зубы
  - Завтрак
    - Сделать завтрак
    - Позавтракать
  - Транспорт
    - Ехать на работу

Использование подобных иерархий чрезвычайно полезно в программировании для определения структуры всей программы. Задача верхнего уровня (например, «Написать доклад о картошке или «От постели к работе») становится функцией main() (так как это основная проблема, которую нужно решить). Подзадачи становятся функциями в программе.

Шаг №4: Определите последовательность событий

Теперь, когда ваша программа имеет структуру, пришло время ответить на вопрос: «А как же связать все эти пункты вместе?». Первый шаг заключается в определении последовательности событий. Например, когда вы просыпаетесь утром, в какой последовательности вы выполняете вышеперечисленные дела? Скорее всего, в следующей:

- Встать с постели
- Выбрать одежду
- Принять душ
- Одеться
- Приготовить завтрак
- Позавтракать
- Почистить зубы
- Ехать на работу

Если бы мы создавали калькулятор, то последовательность задания выглядела бы следующим образом:

- Получить первое значение от пользователя
- Получить математическую операцию от пользователя
- Получить второе значение от пользователя
- Вычислить результат
- Вывести результат

Этот список определяет содержимое функции main():

```
int main()
{
    getFirstNum();
    getAskChoice();
    takeNumber();
    getNumber();
    calculateResult();
    updateHistory();
    brushTeeth();
    printNumber();
}
```

Или, в случае с калькулятором:

```
int main()
{
    // Получаем первое значение от пользователя
    getUserInput();
    getUserInput();

    // Получаем математическую операцию от пользователя
    getMathematicalOperation();

    // Получаем второе значение от пользователя
    getUserInput();

    // Вычисляем результат
    calculateNumber();

    // Выводим результат
    printResult();
}
```

Шаг №5: Определите данные ввода и данные вывода на каждом этапе

После определения иерархии задач и последовательности событий, нам нужно определить, какими будут данные ввода и данные вывода на каждом этапе.

Например, первая функция из вышеприведенного примера — getStartingUp(), данных для ввода и с математической операции. При вызове calculateNumber() у нас уже должны быть 3 фактических данных, которые мы будем использовать в качестве параметров функции. Функция calculateNumber() вычисляет значение результата, но не выводит его. Следовательно, нам необходимо вернуть это результат в качестве возвращаемого значения собой. Например, когда вы просматриваете утром, в какой последовательности вы выполняете вышеперечисленные дела?

Учитывая это, прототип функции становится следующим:

```
int calculateResult(int num1, int op, int num2);
```

Шаг №6: Позаботьтесь о деталях

На этом этапе нам нужно реализовать каждый подпункт из нашей иерархии задач. Если вы разбили задачи на достаточно мелкие куски, то выполнить каждый из них будет довольно несложно. Например:

```
int getMathematicalOperation()
{
    std::cout << "Please enter which operator you want (+ or - or * or / or %): ";

    int op;
    std::cin >> op;

    // А что, если пользователь введет неправильный символ?
    // Как мы до сих пор проверяем?

    return op;
}
```

Шаг №7: Соедините данные ввода с данными вывода в программе

И, наконец, последний шаг — это соединение данных ввода с данными вывода. Например, мы можем отправить выходные данные функции calculateResult() во входные данные функции printResult(), чтобы вторая функция могла вывести результат. Чаще всего в таких случаях используются промежуточные переменные для временного хранения результата и его перемещения между функциями. Например:

```
// Получаем первое значение от пользователя, которое используется для первого аргумента функции calculateNumber()
int num1 = getUserInput();
// Получаем второе значение от пользователя, которое используется для второго аргумента функции calculateNumber()
int num2 = getUserInput();

// Получаем математическую операцию от пользователя, которая используется для третьего аргумента функции calculateNumber()
int op = getMathematicalOperation();

// Вычисляем результат и сохраняем его во временной переменной
int result = calculateResult(num1, op, num2);

// Выводим результат
printResult(result);

return 0;
}
```

Согласитесь, что вариант, приведенный выше, читабельнее варианта без использования временных переменных (см. ниже):

```
printResult(calculateResult(num1, op, num2));
```

Рассмотрим готовую версию программы целиком, со структурой и переношенными данными:

```
int getUserInput()
{
    std::cout << "Please enter an integer: ";

    int value;
    std::cin >> value;

    return value;
}

int getMathematicalOperation()
{
    std::cout << "Please enter which operator you want (+ or - or * or / or %): ";

    int op;
    std::cin >> op;

    // А что, если пользователь введет неправильный символ?
    // Как мы до сих пор проверяем?

    return op;
}

int calculateResult(int x, int op, int y)
{
    // Сначала возьмем, операнды не используются для сравнения двух значений

    if (op == '+')
        return x + y; // Если пользователь ввел оператор сложения (+)
    if (op == '-')
        return x - y; // Если пользователь ввел оператор вычитания (-)
    if (op == '*')
        return x * y; // Если пользователь ввел оператор умножения (*)
    if (op == '/')
        return x / y; // Если пользователь ввел оператор деления (/)
    if (op == '%')
        return x % y; // Если пользователь ввел оператор остатка (%)

    // Иначе, мы предполагаем, что пользователь ввел неправильный оператор
    return -1;
}

void printResult(int result)
{
    std::cout << "Your result is: " << result << std::endl;
}

int main()
{
    // Получаем первое значение от пользователя
    int input1 = getUserInput();

    // Получаем математическую операцию от пользователя
    int op = getMathematicalOperation();

    // Получаем второе значение от пользователя
    int input2 = getUserInput();

    // Вычисляем результат и сохраняем его во временной переменной
    int result = calculateResult(input1, op, input2);

    // Выводим результат
    printResult(result);

    return 0;
}
```

Здесь есть несколько концепций, которые мы еще не рассматривали: условное ветвление и использование оператора равенства — для сравнения двух значений. Не беспокойтесь, если вы это не понимаете — мы всё это детально рассмотрим на следующих уроках.

Советы

Пускай ваши первые программы будут простыми. Очень часто новички ставят слишком высокие планки для своих первых более менее серьезных программ. Например, кто-нуш начнёшь игру с графикой, звуком, монстрами, подземельями и прочим, в которую можно будет создавать найденные вещи. Если вы попытаетесь написать что-нибудь подобное в начале вашего пути как программиста, то очень скоро у вас пропадет любое желание программировать. Вместо этого, пускай ваши первые автоматизированные программы будут попроще, например, «Я хочу написать программу, которая отображает бы 20-point на экране».

Добавляйте новый функциональ со временем. Как только вы напишете простенькую программу, которая работает (даже без «бага), то следующим шагом шагом должно стать добавление нового функционала. Например, когда вы сможете отображать 20-point на экране — добавьте персонажа, который сможет ходить по этому полю. После того, как вы сможете ходить — добавьте стены, которые будут препятствовать вашему движению. После того, как у вас будет стена — постройте из них город. После того, как у вас будет город — добавьте персонажей-горожан. При таком подходе на вас не накапливается всё сразу и вы знаете — что конечно, что делать дальше. Шаги — это маленькие-маленькие и т.д.

Фокусируйтесь только на одном задании в определенный промежуток времени. Не пытайтесь сделать всё и сразу, не растыкиваясь на несколько задач одновременно. Сосредоточьтесь на одном. Лучше иметь одно выполненное задание и пять невыполненных, нежели шесть частично выполненных заданий. Если вы рассеиваете свое внимание в нескольких направлениях, то и ошибок будет много.

Тестируйте каждую новую часть кода. Начинающие программисты часто пишут программу за одно присест. Затем, при компиляции кода, получают гопки ошибок. Поэтому, после написания определенной части кода — сразу компилируйте и тестируйте. Если вы сделаете ошибку, будет работать, то вы будете знать, в каждом случае проблемы, и исправить не будет намного легче. Как только вы убедились, что ваш код работает — переходите к написанию следующей части в этом порядке. Тестирование может занять больше времени, но в конечном итоге ваш код будет работать лучше, так как вам нужно.

Большинство новичков пропустят некоторые из этих шагов и советов, так как это не столь захватывающе, как собственно, сам процесс кодинга. Хорошая новость заключается в том, что чем больше вы освоите все эти концепции — тем скорее для вас естественным в процессе разработки вашим программам будет становиться.